

ТУККЕЛЬ Н.И., ШАЛЬТО А.А., ВЕРБА М.Т., LOST X

TELEGRAM-БОТ ТЕКУЩЕЙ ПОГОДЫ

ПРОГРАММНАЯ ДОКУМЕНТАЦИЯ

Используемые технологии: Python 3.10+, aiogram 3.x, aiohttp, python-dotenv, WeatherAPI.com (REST JSON)

Место разработки: ЮЖНО-САХАЛИНСК

Год: 2026

ОГЛАВЛЕНИЕ

1. [Введение](#1-введение)
 2. [Частное техническое задание на подсистему получения погоды](#2-частное-техническое-задание-на-подсистему-получения-погоды)
 3. [Структурная схема программного обеспечения](#3-структурная-схема-программного-обеспечения)
 4. [Перечень и нумерация событий](#4-перечень-и-нумерация-событий)
 5. [Перечень и нумерация входных переменных](#5-перечень-и-нумерация-входных-переменных)
 6. [Перечень и нумерация выходных воздействий](#6-перечень-и-нумерация-выходных-воздействий)
 7. [Пояснения к используемой нотации](#7-пояснения-к-используемой-нотации)
 8. [Системозависимая часть](#8-системозависимая-часть)
 9. [Исходные тексты системозависимой части](#9-исходные-тексты-системозависимой-части)
 10. [Протоколы функционирования](#10-протоколы-функционирования)
 11. [Требования к программному обеспечению](#11-требования-к-программному-обеспечению)
 12. [Требования к программной документации](#12-требования-к-программной-документации)
 13. [Технико-экономические показатели](#13-технико-экономические-показатели)
 14. [Стадии и этапы разработки](#14-стадии-и-этапы-разработки)
 15. [Порядок контроля и приёмки](#15-порядок-контроля-и-приёмки)
 16. [Установка и эксплуатация](#16-установка-и-эксплуатация)
-

1. ВВЕДЕНИЕ

Настоящий документ представляет собой проектную документацию на программное обеспечение «Telegram-бот текущей погоды» (далее — Система, Бот).

Система предназначена для предоставления пользователям мессенджера Telegram актуальной информации о погоде в заданной точке Земли. Пользователь может:

- отправить геолокацию (встроенное вложение Telegram);

- ввести текстовый запрос — название города, района, страны или произвольный адрес на языке, поддерживаемом сервисом WeatherAPI.com.

Бот обращается к внешнему REST API, получает JSON с текущими метеоданными, форматирует ответ на русском языке и отправляет сообщение в чат с разметкой HTML.

Документация разработана в соответствии с требованиями к курсовому проектированию и демонстрирует применение автоматного подхода (SWITCH-технологии) для описания логики обработки сообщений, запросов к API и формирования ответа.

Состав репозитория:

Компонент	Назначение
<code>bot.py</code>	Единственный исполняемый модуль: инициализация, обработчики, запуск polling
<code>.env</code>	Секреты: токен Telegram и ключ WeatherAPI (не включается в git)
<code>.env.example</code>	Шаблон переменных окружения
<code>requirements.txt</code>	Зависимости Python
<code>doc/</code>	Программная документация и блок-схемы (PNG)

2. ЧАСТНОЕ ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПОДСИСТЕМУ ПОЛУЧЕНИЯ ПОГОДЫ

2.1. Назначение подсистемы

Подсистема получения и отображения погоды является составной частью программного комплекса и обеспечивает:

- приём входящих обновлений от Telegram Bot API через long polling (aiogram);
- маршрутизацию сообщений по типу (команда `/start`, геолокация, текст);
- формирование HTTP GET-запроса к `http://api.weatherapi.com/v1/current.json`;
- разбор JSON-ответа и построение пользовательского сообщения;
- индикацию «печатает...» (`send_chat_action`) на время ожидания ответа API;
- обработку ошибок сети, HTTP и структуры данных.

2.2. Функциональные требования

2.2.1. Команда приветствия

При получении команды `/start` бот обязан отправить текстовое приветствие с инструкцией по использованию (геолокация или название места).

2.2.2. Запрос по геолокации

При получении объекта Message с заполненным полем `location` подсистема должна:

1. Извлечь `latitude` и `longitude`.
2. Передать в API параметр `q` в формате "`<lat>,<lon>`".
3. При успешном ответе (HTTP 200) вызвать функцию `send_weather_info`.

2.2.3. Запрос по тексту

При получении текстового сообщения, не являющегося командой (строка не начинается с `/` после `strip()`), подсистема должна передать содержимое текста в параметр `q` API.

Пустые строки и команды игнорируются (обработчик завершается без ответа).

2.2.4. Формат ответа пользователю

Сообщение должно содержать (при успешном разборе):

- название населённого пункта и страну;
- температуру °C и «ощущается как»;
- текстовое описание условий (облачность, осадки и т.д.);
- влажность %;
- скорость и направление ветра;
- метку времени последнего обновления данных API.

Специальные символы HTML в полях из API экранируются функцией `html.escape`.

2.2.5. Обработка ошибок

Ситуация	Поведение
HTTP ≠ 200 (геолокация)	Сообщение «Не удалось получить данные о погоде»
HTTP ≠ 200 (текст)	Сообщение с <code>error.message</code> из JSON API, если доступно
Исключение <code>aiohttp/сеть</code>	Лог ERROR + «Произошла ошибка при обращении к сервису погоды»
<code>KeyError</code> при разборе JSON	Лог ERROR + «Не удалось обработать данные о погоде»

2.3. Требования к производительности

- Время ответа пользователю определяется задержкой WeatherAPI и сети; целевое время — не более 5 с при стабильном интернете.
- Бот рассчитан на однопоточную асинхронную модель `aiogram`; параллельная обработка множества чатов обеспечивается `event loop asyncio`.
- Допускается обработка неограниченного числа пользователей в пределах лимитов Telegram и тарифа WeatherAPI.

2.4. Требования к конфигурированию

- `BOT_TOKEN` — токен бота от `[@BotFather](https://t.me/BotFather)`.
- `WEATHER_API_KEY` — ключ с `[weatherapi.com](https://www.weatherapi.com/)`.
- Параметр `lang=ru` задан в коде для локализации текстовых описаний погоды в ответе API.

2.5. Ограничения

- Используется только endpoint current (текущая погода), без прогноза на несколько дней.
- Запросы к API выполняются по HTTP (не HTTPS) — как в исходной реализации; при развёртывании в production рекомендуется <https://api.weatherapi.com>.
- Отсутствует персистентное хранилище истории запросов и избранных городов.

3. СТРУКТУРНАЯ СХЕМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

3.1. Общая структурная схема

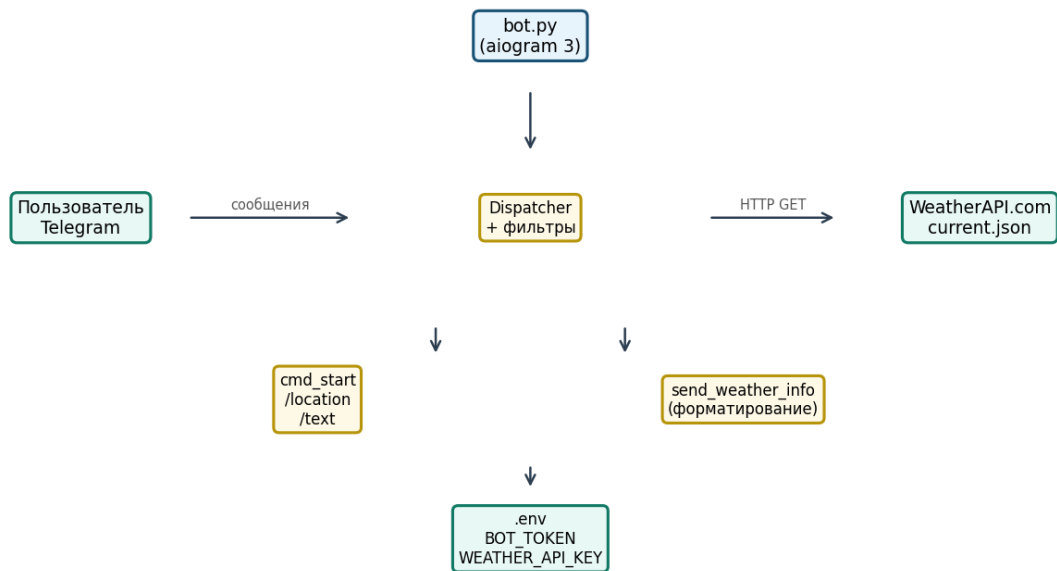
Программа построена по монолитному одномодульному принципу: весь прикладной код сосредоточен в `bot.py`. Логически выделяются следующие компоненты:

Модуль / компонент	Файл	Ответственность
Загрузка конфигурации	<code>bot.py</code> (строки 9–12)	<code>load_dotenv()</code> , чтение <code>BOT_TOKEN</code> , <code>WEATHER_API_KEY</code>
Инициализация Telegram	<code>bot.py</code> (17–19)	<code>Bot</code> , <code>Dispatcher</code>
Обработчик <code>/start</code>	<code>cmd_start</code>	Приветствие
Обработчик геолокации	<code>handle_location</code>	Запрос API по координатам
Обработчик текста	<code>handle_text</code>	Запрос API по строке
Форматирование ответа	<code>send_weather_info</code>	Парсинг JSON → HTML сообщение
Точка входа	<code>main()</code>	<code>dp.start_polling(bot)</code>

Рисунок 3.1 — Общая структурная схема:

Общая структурная схема

Рис. 3.1 — Общая структурная схема ПО «Telegram-бот погоды»



3.2. Порядок взаимодействия частей подсистемы

1. Пользователь отправляет сообщение в Telegram.
2. Telegram Bot API доставляет update процессу бота (long polling).
3. Dispatcher выбирает первый подходящий зарегистрированный handler:
 - `Command("start")` — высший приоритет для `/start`;
 - `lambda: message.location is not None` — для геолокации;
 - `F.text` — для остального текста.
4. Выбранный handler вызывает `send_chat_action(typing)`.
5. Создаётся `aiohttp.ClientSession`, выполняется `GET current.json` с `key` и `q`.
6. При статусе 200 JSON передаётся в `send_weather_info`.
7. Пользователь получает HTML-сообщение в том же чате.

Рисунок 3.2 — Диаграмма взаимодействия:

Порядок взаимодействия

Рис. 3.2 — Порядок взаимодействия модулей (основной сценарий)

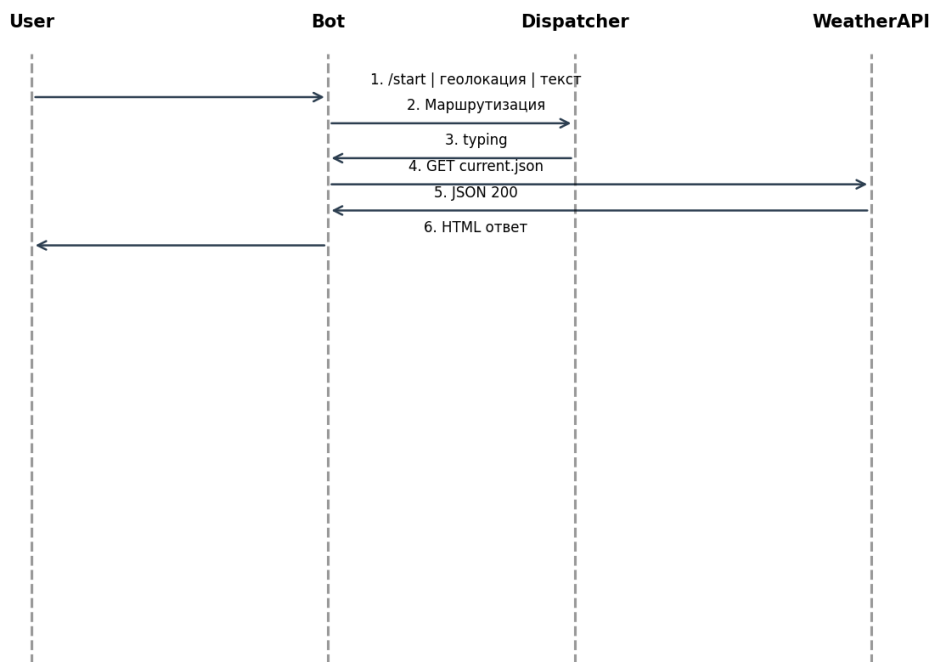
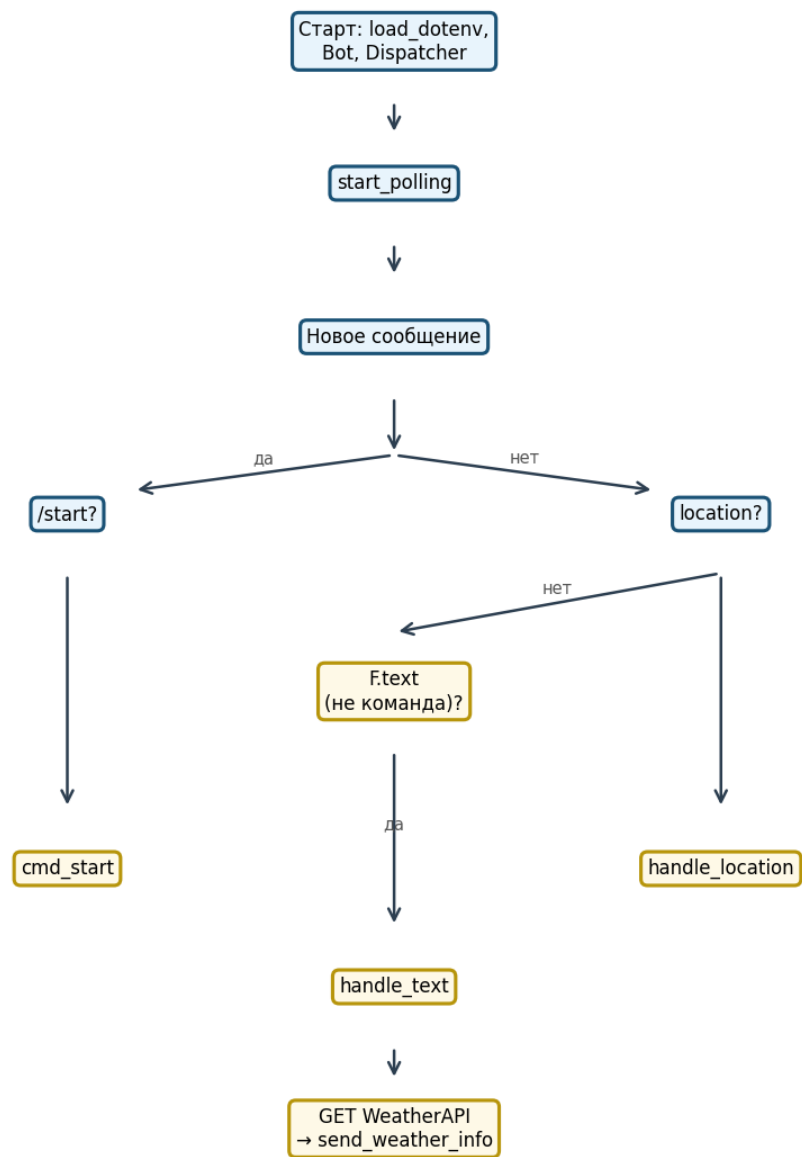


Рисунок 3.3 — Блок-схема обработки сообщений:

Главный поток

Рис. 8.0 — Блок-схема главного цикла и обработки сообщений



4. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ

События генерируются средой (Telegram, сеть, таймер polling) и обрабатываются логикой bot . py и автоматами A0–A2.

Номер	Обозначение	Описание
m1	EVT_START_CMD	Получена команда /start
m2	EVT_LOCATION	Получена геолокация

Номер	Обозначение	Описание
m3	EVT_TEXT	Получено текстовое сообщение (не команда)
m4	EVT_OTHER	Прочие типы сообщений (игнор)
m10	EVT_SESSION_OPEN	Открыта сессия aiohttp
m11	EVT_HTTP_200	Ответ API со статусом 200
m12	EVT_HTTP_ERROR	Ответ API с ошибочным статусом
m13	EVT_JSON_OK	JSON успешно разобран
m14	EVT_NETWORK_EX	Исключение при HTTP-запросе
m18	EVT_POLL_TICK	Очередной цикл long polling
m20	EVT_PARSE_OK	Все поля JSON для ответа найдены
m21	EVT_PARSE_KEYERR	KeyError при извлечении полей
m22	EVT_MSG_BUILT	Строка HTML сформирована
m23	EVT_MSG_SENT	message.answer выполнен
g50	EVT_TYPING	Отправлен chat action «typing»

5. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ

Номер	Обозначение	Тип	Описание	
g1	BOT_TOKEN	str	Токен Telegram Bot API	
g2	WEATHER_API_KEY	str	Ключ WeatherAPI	
g41	message.text	`str`	None`	Текст сообщения пользователя
g42	message.location	`Location`	None`	Координаты
g43	message.chat.id	int	Идентификатор чата	
g92	lat, lon	float	Широта и долгота	
g93	query	str	Строка поиска для параметра q	
g94	resp.status	int	HTTP-код ответа	

Номер	Обозначение	Тип	Описание	
g95	data	dict	Тело JSON ответа API	

Параметры запроса к WeatherAPI (формируются в коде):

```
params = {"key": WEATHER_API_KEY, "q": "<■■■■■■■■ ■■■■ "lat,lon">, "lang": "ru"}
```

6. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ

Номер	Обозначение	Описание
m63	OUT_WELCOME	Текст приветствия /start
m89	OUT_WEATHER_HTML	Сообщение с погодой (parse_mode HTML)
m90	OUT_ERROR_GENERIC	Общая ошибка API/сети
m91	OUT_ERROR_API_MSG	Текст ошибки из поля error.message
g268	OUT_LOG_ERROR	Запись в журнал logging
g92	OUT_CHAT_ACTION	Индикатор «печатает...»

7. ПОЯСНЕНИЯ К ИСПОЛЬЗУЕМОЙ НОТАЦИИ

7.1. Нотация графов переходов

- Состояния обозначаются S0, S1, ... или префиксом автомата (W0, R0).
- События — по таблице раздела 4; на рёбрах графа указано условие перехода.
- Начальное состояние — с наименьшим индексом (например, S0).
- После обработки запроса автоматы A0–A2 возвращаются в состояние ожидания следующего сообщения (концептуально S0 / W0 / R0).

7.2. Шаблон реализации автомата в Python (aiogram)

В отличие от явного `switch(state)` в процедурном коде, aiogram использует декларативную маршрутизацию: каждый автомат «ожидания» реализован отдельным `@dp.message`-обработчиком. Состояние хранится неявно — в том, какой handler сработал.

Псевдокод эквивалента A0:

```
■■■ ■■■■■■■■■ message:
    ■■■■ ■■■■■■■ start → cmd_start
    ■■■■ ■■■■ location → handle_location
    ■■■■ ■■■■ text ■ ■■ ■■■■■■■ → handle_text
    ■■■■ → ■■■■■■
```

8. СИСТЕМОНЕЗАВИСИМАЯ ЧАСТЬ

8.1. Автомат A0 — маршрутизация входящего сообщения

8.1.1. Словесное описание

Автомат A0 описывает выбор ветки обработки при поступлении update типа message. Dispatcher aiogram проверяет фильтры в порядке регистрации. Для данного проекта порядок регистрации: /start → геолокация → F.text.

- S0 (Ожидание): нет активной внутренней транзакции; следующее сообщение классифицируется.
- S1: выполняется cmd_start, переход в S0 после ответа.
- S2: выполняется handle_location → цепочка A1 → A2.
- S3: выполняется handle_text → A1 → A2.
- S4: сообщение не подходит ни под один фильтр — ответ не отправляется.

8.1.2. Схема связей и граф переходов

Автомат A0

Рис. 8.1.2 — Автомат A0: маршрутизация входящего сообщения



8.1.3. Текст функции, реализующей автомат (фрагменты)

Регистрация обработчиков (эквивалент графа A0):

```

@dp.message(Command("start"))
async def cmd_start(message: types.Message): ...

@dp.message(lambda message: message.location is not None)
async def handle_location(message: types.Message): ...

@dp.message(F.text)
async def handle_text(message: types.Message):

```

```
if not query or query.startswith('/'):
    return # ■■■■■■■■ ■ S4 - ■■■■■■
```

8.2. Автомат A1 — запрос к WeatherAPI

8.2.1. Словесное описание

Автомат A1 описывает жизненный цикл одного HTTP-запроса:

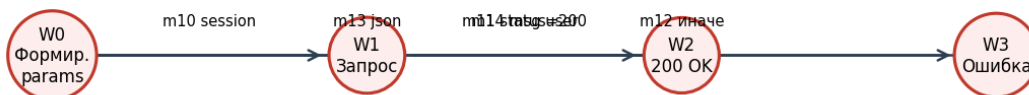
1. W0: формирование url и params (q, key, lang).
2. W1: выполнение session.get; ожидание ответа.
3. W2: при status == 200 — чтение JSON, вызов send_weather_info (запуск A2).
4. W3: при ином статусе или исключении — сообщение об ошибке пользователю, запись в log.

Состояния W0–W3 не сохраняются между сообщениями: каждый запрос — новый экземпляр цикла.

8.2.2. Схема связей и граф переходов

Автомат A1

Рис. 8.2.2 — Автомат A1: запрос к WeatherAPI



8.2.3. Текст функции, реализующей автомат

Общий фрагмент для handle_location и handle_text:

```
url = "http://api.weatherapi.com/v1/current.json"
params = {"key": WEATHER_API_KEY, "q": ..., "lang": "ru"}

async with aiohttp.ClientSession() as session:
    try:
        async with session.get(url, params=params) as resp:
            if resp.status == 200:
                data = await resp.json()
                await send_weather_info(message, data)
```

```

else:
    # ████████ W3
    ...
except Exception as e:
    logging.error(...)
    await message.answer("██ ██████████ ████████...")

```

8.3. Автомат A2 — формирование и отправка ответа

8.3.1. Словесное описание

Автомат A2 работает с уже полученным словарём `data`:

- R0: извлечение вложенных ключей `location`, `current`.
- R1: экранирование строк HTML, сбор многострочного `text`.
- R2: `await message.answer(text, parse_mode="HTML")`.
- R3: при `KeyError` — предупреждение пользователю и лог.

8.3.2. Схема связей и граф переходов

Автомат A2

Рис. 8.3.2 — Автомат A2: формирование и отправка ответа



8.3.3. Текст функции, реализующей автомат

```

async def send_weather_info(message: types.Message, data: dict):
    try:
        location = escape(data["location"]["name"])
        country = escape(data["location"]["country"])
        temp_c = data["current"]["temp_c"]
        condition = escape(data["current"]["condition"]["text"])
        feelslike_c = data["current"]["feelslike_c"]
        humidity = data["current"]["humidity"]
        wind_kph = data["current"]["wind_kph"]

```

```

wind_dir = escape(data["current"]["wind_dir"])
last_updated = escape(data["current"]["last_updated"])
text = (f"■ <b>{location}, {country}</b>\n" ...)
await message.answer(text, parse_mode="HTML")
except KeyError as e:
    logging.error(f"■■■■■■■■ ■■■■■■■■ ■■■■■■■■: {e}")
    await message.answer("■■ ■■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■■ ■ ■■■■■■■.")

```

9. ИСХОДНЫЕ ТЕКСТЫ СИСТЕМОНЕЗАВИСИМОЙ ЧАСТИ

Полный актуальный исходный код расположен в корне репозитория: `./bot.py` (122 строки).

9.1. Структура файла `bot.py`

Строки	Содержание
1–7	Импорты
9–15	Загрузка <code>.env</code> , логирование
17–19	<code>Bot</code> , <code>Dispatcher</code>
22–27	<code>cmd_start</code>
30–53	<code>handle_location</code>
56–88	<code>handle_text</code>
90–114	<code>send_weather_info</code>
117–122	<code>main</code> , <code>asyncio.run</code>

9.2. Внешний API — структура ответа `current.json` (используемые поля)

```

{
  "location": { "name": "...", "country": "..." },
  "current": {
    "temp_c": 0.0,
    "feelslike_c": 0.0,
    "condition": { "text": "..." },
    "humidity": 0,
    "wind_kph": 0.0,
    "wind_dir": "N",
    "last_updated": "2026-05-27 12:00"
  }
}

```

9.3. Конфигурация окружения (`.env.example`)

```

BOT_TOKEN=your_telegram_bot_token
WEATHER_API_KEY=your_weatherapi_key

```

10. ПРОТОКОЛЫ ФУНКЦИОНИРОВАНИЯ

10.1. Диагностирующий протокол (полный)

№	Действие	Ожидаемый результат	Факт	Примечание
1	Установить зависимости <code>pip install -r requirements.txt</code>	Без ошибок		
2	Создать <code>.env</code> по образцу	Файл с двумя ключами		
3	Запустить <code>python bot.py</code>	В логе INFO, нет traceback		
4	Отправить <code>/start</code>	Приветственный текст		
5	Отправить геолокацию	Сообщение с погодой, HTML		
6	Отправить «Москва»	Погода для Москвы		
7	Отправить «zzzzinvalidcity123»	Сообщение об ошибке API		
8	Остановить бот (Ctrl+C)	Процесс завершён		

10.2. Проверяющий протокол (короткий)

№	Проверка	OK / FAIL
1	<code>/start</code> отвечает	
2	Геолокация → погода	
3	Город текстом → погода	

11. ТРЕБОВАНИЯ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ

11.1. Функциональные характеристики

- Поддержка Telegram-клиентов с отправкой геолокации и текста.
- Локализация описания погоды через `lang=ru`.
- Безопасный вывод: экранирование HTML в пользовательских данных API.

11.2. Надёжность

- Ошибки сети не приводят к падению процесса.
- Некорректный JSON/структура обрабатываются в `send_weather_info` и в ветках HTTP.

11.3. Состав технических средств

Компонент	Минимальные требования
ОС	Windows 10+, Linux, macOS
Python	3.10 и выше
Сеть	Исходящий доступ к <code>api.telegram.org</code> , <code>api.weatherapi.com</code>
RAM	≥ 128 МБ для процесса бота

11.4. Информационная совместимость

- Telegram Bot API (актуальная версия на момент эксплуатации).
- WeatherAPI REST v1, endpoint `/v1/current.json`.

12. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

Документация должна включать:

1. Настоящий файл `DOCUMENTATION.md`.
2. Комплект PNG-блок-схем в каталоге `doc/diagrams/` (6 рисунков).
3. Описание автоматов A0, A1, A2 с графами переходов.
4. Протоколы испытаний (раздел 10).
5. Инструкцию по установке (раздел 16).

13. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

Показатель	Значение
Объём кода	~120 строк Python
Число внешних зависимостей	3 (aiogram, aiohttp, python-dotenv)
Стоимость инфраструктуры	VPS опционален; бесплатный tier WeatherAPI для учебных нагрузок
Трудозатраты разработки	1 модуль, низкая сложность сопровождения

14. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

Этап	Содержание	Результат
1	Анализ ТЗ, выбор API	Выбор WeatherAPI + aiogram 3
2	Реализация handlers	<code>bot.py</code>
3	Тестирование в Telegram	Протокол раздела 10

Этап	Содержание	Результат
4	Документирование	Каталог doc/
5	Подготовка к сдаче	.gitignore, requirements.txt

15. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ

Программа считается принятой при:

1. Успешном прохождении проверяющего протокола (раздел 10.2).
2. Наличии актуальной документации в doc/.
3. Отсутствии незакоммиченных секретов в репозитории (файл .env в .gitignore).

Ответственный за приёмку — руководитель курсового проекта / заказчик учебного задания.

16. УСТАНОВКА И ЭКСПЛУАТАЦИЯ

16.1. Установка

```
python -m venv venv
venv\Scripts\activate          # Windows
pip install -r requirements.txt
copy .env.example .env         # ██████████ ████████
python bot.py
```

16.2. Переменные окружения

Переменная	Описание
BOT_TOKEN	Токен от BotFather
WEATHER_API_KEY	Ключ WeatherAPI

16.3. Эксплуатация

- Процесс должен работать непрерывно для обработки сообщений (polling).
- Рекомендуется запуск как systemd-сервис или через docker на сервере.
- Мониторинг — по логам logging уровня INFO/ERROR.

16.4. Каталог блок-схем

Файл	Содержание
diagrams/01_struct_general.png	Рис. 3.1 — структурная схема
diagrams/02_interaction_sequence.png	Рис. 3.2 — взаимодействие
diagrams/03_main_flowchart.png	Рис. 8.0 — блок-схема обработки
diagrams/04_automaton_A0_routing.png	Рис. 8.1.2 — автомат A0

Файл	Содержание
diagrams/05_automaton_A1_api.png	Рис. 8.2.2 — автомат A1
diagrams/06_automaton_A2_response.png	Рис. 8.3.2 — автомат A2

Конец документа.